LINEAR REGRESSION / SOURCE CODE

React + TypeScript (TSX) | A4 가로 인쇄용 | 원본 200 줄 전체 수록

읽는 법 왼쪽 숫자 = 원본 줄 번호 · 파랑 = 키워드/함수 · 초록 = 문자열 · 회색 = 주석 · 보라 = 타입/컴포넌트

01 데이터와 수학 함수 Point, Weights, predict, cost, gradient

```tsx
import { useEffect, useMemo, useRef, useState, type MouseEvent } from 'react';

type Point = { id: number; x: number; y: number };
type Weights = { w: number; b: number };
const initialData: Point[] = [
  { id: 1, x: -.75, y: 0 }, { id: 2, x: -.60, y: -.40 },
  { id: 3, x: -.51, y: .02 }, { id: 4, x: -.30, y: 0 },
  { id: 5, x: -.15, y: .33 }, { id: 6, x: .15, y: .50 },
  { id: 7, x: .45, y: .50 }, { id: 8, x: .53, y: .83 },
  { id: 9, x: .85, y: .70 }, { id: 10, x: .98, y: .90 },
];
const predict = (x: number, w: number, b: number) => w * x + b;
// 원본 시뮬레이터와 유사한 비용: J = (1/2) * Σ(예측 - 실제)^2
const cost = (data: Point[], w: number, b: number) =>
  data.reduce((sum, p) => sum + (predict(p.x, w, b) - p.y) ** 2, 0) / 2;
// J 의 편미분: ∂J/∂w = Σ(error*x), ∂J/∂b = Σ(error)
const gradient = (data: Point[], w: number, b: number) =>
  data.reduce((g, p) => {
    const error = predict(p.x, w, b) - p.y;
    return { dw: g.dw + error * p.x, db: g.db + error };
  }, { dw: 0, db: 0 });

const fmt = (n: number, places = 3) => (n >= 0 ? '+' : '') + n.toFixed(places);
const clamp = (n: number, lo: number, hi: number) => Math.min(hi, Math.max(lo, n));

```

02 회귀 그래프 RegressionPlot / SVG 렌더링

```
function RegressionPlot({ data, weights, onAdd }: { data: Point[]; weights: Weights; onAdd: (x: number, y: number) => void }) {
  const sx = (x: number) => 58 + (x + 1.4) / 2.8 * 430;
  const sy = (y: number) => 344 - (y + 1.4) / 2.8 * 288;
  const xMin = Math.min(-1.2, ...data.map(p => p.x - .15));
  const xMax = Math.max(1.2, ...data.map(p => p.x + .15));
  const minY = Math.min(-1.3, ...data.map(p => p.y - .15), predict(xMin, weights.w, weights.b));
  const maxY = Math.max(1.3, ...data.map(p => p.y + .15), predict(xMax, weights.w, weights.b));
  const rx = (x: number) => 58 + (x - xMin) / (xMax - xMin) * 430;
  const ry = (y: number) => 344 - (y - minY) / (maxY - minY) * 288;
  const xticks = [-2, -1, 0, 1, 2].filter(t => t >= xMin && t <= xMax);
  const yticks = [-2, -1, 0, 1, 2, 3].filter(t => t >= minY && t <= maxY);
  const svgRef = useRef<SVGSVGElement>(null);
  const clickPlot = (e: MouseEvent<SVGRectElement>) => {
    const svg = svgRef.current;
    if (!svg) return;
    const point = svg.createSVGPoint(); point.x = e.clientX; point.y = e.clientY;
    const matrix = svg.getScreenCTM(); if (!matrix) return;
    const local = point.matrixTransform(matrix.inverse());
    const x = xMin + (local.x - 58) / 430 * (xMax - xMin);
    const y = minY + (344 - local.y) / 288 * (maxY - minY);
    onAdd(Number(x.toFixed(2)), Number(y.toFixed(2)));
  };
  void sx; void sy;
  return <svg ref={svgRef} className="plot" viewBox="0 0 525 400" role="img" aria-label="선형회귀 데이터와 예측 직선 그래프">
    <rect x="0" y="0" width="525" height="400" fill="white" />
    {xticks.map(t => <g key={'x'+t}><line x1={rx(t)} x2={rx(t)} y1="56" y2="344" className="gridline"/><text x={rx(t)} y="363"
textAnchor="middle" className="axis-tick">{t}</text></g>)}
    {yticks.map(t => <g key={'y'+t}><line x1="58" x2="488" y1={ry(t)} y2={ry(t)} className="gridline"/><text x="48" y={ry(t)+4}
textAnchor="end" className="axis-tick">{t}</text></g>)}
    {xMin <= 0 && xMax >= 0 && <line x1={rx(0)} x2={rx(0)} y1="56" y2="344" className="zero-axis"/>}
```

```
      {minY <= 0 && maxY >= 0 && <line x1="58" x2="488" y1={ry(0)} y2={ry(0)} className="zero-axis"/>}
      <line x1={rx(xMin)} y1={ry(predict(xMin, weights.w, weights.b))} x2={rx(xMax)} y2={ry(predict(xMax, weights.w, weights.b))}
stroke="#1984c5" strokeWidth="2.6"/>
      {data.map(p => <circle key={p.id} cx={rx(p.x)} cy={ry(p.y)} r="4" fill="#ff790f" stroke="white" strokeWidth=".6"/>)}
      <rect x="68" y="50" width="135" height="48" fill="#eee" opacity=".97"/>
      <line x1="78" y1="66" x2="106" y2="66" stroke="#1984c5" strokeWidth="2"/><text x="113" y="70"
className="legend">hypothesis</text>
      <circle cx="91" cy="83" r="3.4" fill="#ff790f"/><text x="113" y="87" className="legend">data</text>
      <text x="273" y="390" textAnchor="middle" className="axis-label">X Axis</text>
      <text x="18" y="204" textAnchor="middle" transform="rotate(-90 18 204)" className="axis-label">Y Axis</text>
      <rect x="58" y="56" width="430" height="288" fill="transparent" className="plot-click" onClick={clickPlot}>
        <title>그래프를 클릭하면 데이터 점이 추가돼요</title>
      </rect>
    </svg>;
  }

```

03 비용함수 그래프 색상 팔레트와 ContourPlot

```
  const palette = ['#173b83','#124f98','#0968ac','#0b80b4','#399cac','#77ad95','#b0c466','#e5d044','#f3b43c','#f59931','#f1722b','#e95
824','#d93320'];
  function colorFor(v: number) {
    const idx = clamp(Math.floor(v / 1.15), 0, palette.length - 1);
    return palette[idx];
  }
  function ContourPlot({ data, weights }: { data: Point[]; weights: Weights }) {
    const canvas = useRef<HTMLCanvasElement>(null);
    const [showPath, setShowPath] = useState(true);
    useEffect(() => {
      const el = canvas.current;
      if (!el) return;
      const context = el.getContext('2d');
```

```
      if (!context) return;
      const width = el.width, height = el.height;
      const im = context.createImageData(width, height);
      const [min, max] = [-1, 1];
      for (let py = 0; py < height; py++) for (let px = 0; px < width; px++) {
        const w = min + px / (width - 1) * (max - min);
        const b = max - py / (height - 1) * (max - min);
        const hex = colorFor(cost(data, w, b));
        const off = (py * width + px) * 4;
        im.data[off] = parseInt(hex.slice(1,3), 16);
        im.data[off+1] = parseInt(hex.slice(3,5), 16);
        im.data[off+2] = parseInt(hex.slice(5,7), 16);
        im.data[off+3] = 255;
      }
      context.putImageData(im, 0, 0);
    }, [data]);
    const x = 43 + (weights.w + 1) / 2 * 360;
    const y = 350 - (weights.b + 1) / 2 * 290;
    return <div className="contour-container">
      <div className="contour-topline"><span>COST : {cost(data, weights.w, weights.b).toFixed(3)}</span><button className="small-link"
onClick={() => setShowPath(p => !p)}>{showPath ? '현재 위치 숨기기' : '현재 위치 표시'}</button></div>
      <svg viewBox="0 0 510 410" className="plot" aria-label="가중치와 편향에 따른 비용함수 등고선">
        <foreignObject x="43" y="60" width="360" height="290"><canvas ref={canvas} width="360" height="290" style={{ width: '360px',
height: '290px', display: 'block' }}/></foreignObject>
        {[-1,-.5,0,.5,1].map(t => <g key={t}><text x={43+(t+1)/2*360} y="366" textAnchor="middle"
className="axis-tick">{t}</text><text x="34" y={350-(t+1)/2*290+4} textAnchor="end" className="axis-tick">{t}</text></g>)}
        <text x="223" y="393" textAnchor="middle" className="axis-label">w1 Axis</text>
        <text x="12" y="207" textAnchor="middle" transform="rotate(-90 12 207)" className="axis-label">w0 Axis</text>
        {palette.slice().reverse().map((c,i) => <rect key={i} x="422" y={68+i*21} width="26" height="21" fill={c}/>)}
        {[0,2,4,6,8,10,12,14].map(t => <text key={t} x="455" y={348-t/14*280} className="axis-tick">{t}</text>)}
        {showPath && weights.w >= -1 && weights.w <= 1 && weights.b >= -1 && weights.b <= 1 && <g><circle cx={x} cy={y} r="7"
```

```
fill="none" stroke="white" strokeWidth="2.5"/><circle cx={x} cy={y} r="4" fill="#fa3b2f" stroke="white" strokeWidth="1"/></g>}
      </svg>
      <p className="helper">색상이 푸를수록 비용이 작아요. 빨간 점은 현재 w₁, w₀ 입니다.</p>
    </div>;
  }

```

04 조절 다이얼 Knob 컴포넌트

```
function Knob({ label, value, onChange }: { label: string; value: number; onChange: (value: number) => void }) {
  const ratio = (value + 1.5) / 3;
  const angle = -140 + ratio * 280;
  return <div className="knob-field">
    <label className="knob-label">{label}: {value.toFixed(3)}</label>
    <div className="knob" aria-hidden="true"><div className="knob-dot" style={{ transform: `translate(-50%, -50%)
rotate(${angle}deg) translateY(-36px)` }}/></div>
    <input aria-label={label} type="range" min="-1.5" max="1.5" step="0.001" value={value} onChange={e =>
onChange(Number(e.target.value))}/>
  </div>;
}

```

05 앱 상태와 학습 계산 useState, 경사하강법, 최소제곱법

```
function App() {
  const [data, setData] = useState<Point[]>(initialData);
  const nextId = useRef(11);
  const [weights, setWeights] = useState<Weights>({w: 0, b: 0});
  const [learningRate, setLearningRate] = useState(.04);
  const [steps, setSteps] = useState(0);
  const [running, setRunning] = useState(false);
  const [addX, setAddX] = useState('0');
  const [addY, setAddY] = useState('0');
```

```
  const [showCost, setShowCost] = useState(true);
  const [model, setModel] = useState('Linear');
  const totalCost = useMemo(() => cost(data, weights.w, weights.b), [data, weights]);
  const changeData = (id: number, field: 'x' | 'y', value: number) => setData(prev => prev.map(p => p.id === id ? { ...p, [field]:
value } : p));
  const addPoint = (x: number, y: number) => { setRunning(false); setData(prev => [...prev, { id: nextId.current++, x, y }]); };
  const reset = () => { setRunning(false); setWeights({ w: 0, b: 0 }); setSteps(0); };
  useEffect(() => {
    if (!running) return;
    const timer = window.setInterval(() => {
      setWeights(prev => {
        const g = gradient(data, prev.w, prev.b);
        const newWeights = {w: prev.w-learningRate*g.dw, b: prev.b-learningRate*g.db};
        if (!Number.isFinite(newWeights.w) || !Number.isFinite(newWeights.b) || Math.abs(newWeights.w)>1e6 ||
Math.abs(newWeights.b)>1e6) {
          setRunning(false); return prev;
        }
        return newWeights;
      });
      setSteps(n => n+1);
    }, 90);
    return () => window.clearInterval(timer);
  }, [running, data, learningRate]);
  const solve = () => {
    if (data.length < 2) return;
    const meanX = data.reduce((s,p)=>s+p.x,0)/data.length;
    const meanY = data.reduce((s,p)=>s+p.y,0)/data.length;
    const numerator = data.reduce((s,p)=>s+(p.x-meanX)*(p.y-meanY),0);
    const denominator = data.reduce((s,p)=>s+(p.x-meanX)**2,0);
    if (denominator === 0) { alert('모든 x 값이 같으면 기울기를 구할 수 없어요.'); return; }
    const w = numerator/denominator;
```

```
    setRunning(false); setWeights({w,b:meanY-w*meanX});
  };
```

06 화면 구성 및 내보내기 UI JSX / export default

```
  return <div className="page">
    <header className="topbar"><div><span className="eyebrow">DEEP LEARNING LAB</span><h1>Linear Regression Visualizer</h1></div><p>선형회귀와 경사하강법을 직접 실험해보세요.</p></header>
    <main>
      <div className="graphs">
        <section className="panel"><div className="panel-title">Linear Regression</div><RegressionPlot data={data} weights={weights} onAdd={addPoint}/></section>
        <section className="panel"><div className="panel-title">Cost Function</div>{showCost ? <ContourPlot data={data} weights={weights}/> : <div className="empty-cost">Cost Function 표시가 꺼져 있습니다.</div>}</section>
      </div>
      <div className="bottom-layout">
        <aside className="controls">
          <div className="knobs"><Knob label="w₁" value={weights.w} onChange={v=>{setRunning(false);setWeights(p=>({...p,w:v}));}}/><Knob label="w₀" value={weights.b} onChange={v=>{setRunning(false);setWeights(p=>({...p,b:v}));}}/></div>
          <label className="control-label" htmlFor="model">Model</label>
          <select id="model" value={model} onChange={e=>setModel(e.target.value)}><option>Linear</option></select>
          <label className="checkbox-line"><input type="checkbox" checked={showCost} onChange={e=>setShowCost(e.target.checked)}/> cost 표시</label>
          <label className="control-label" htmlFor="rate">Learning rate <strong>{learningRate.toFixed(3)}</strong></label>
          <input id="rate" className="full-slider" type="range" min="0.001" max="0.2" step="0.001" value={learningRate} onChange={e=>setLearningRate(Number(e.target.value))}/>
          <div className="actions"><button className="btn btn-blue" onClick={()=>setRunning(true)} disabled={running || data.length===0}>Learn</button><button className="btn btn-red" onClick={()=>setRunning(false)}>Stop</button><button className="btn btn-muted" onClick={reset}>Reset</button></div>
          <div className="stats"><span>학습 횟수: <strong>{steps}</strong></span><span>현재 비용: <strong>{totalCost.toFixed(4)}</strong></span></div>
          <button className="solve" onClick={solve}>최소제곱법으로 즉시 계산</button>
```

```
            <p className="hint">다이얼 아래 슬라이더로 w₁, w₀ 을 조절하세요. 그래프를 클릭해 점을 추가할 수도 있어요.</p>
          </aside>
          <section className="dataset">
            <div className="data-heading"><strong>Dataset</strong><span>{data.length}개 데이터</span></div>
            <div className="table-scroll"><table><thead><tr><th></th>{data.map((p,i)=><th key={p.id}>D{i+1}</th>)}</tr></thead><tbody>
              <tr><th className="orange">x</th>{data.map(p=><td key={p.id}><input aria-label={'x '+p.id} className="table-input orange" type="number" step="any" value={p.x} onChange={e=>changeData(p.id,'x', Number(e.target.value))}/></td>)}</tr>
              <tr><th className="orange">y</th>{data.map(p=><td key={p.id}><input aria-label={'y '+p.id} className="table-input orange" type="number" step="any" value={p.y} onChange={e=>changeData(p.id,'y', Number(e.target.value))}/></td>)}</tr>
              <tr><th className="blue">h</th>{data.map(p=><td key={p.id} className="blue">{fmt(predict(p.x,weights.w,weights.b))}</td>)}</tr>
              <tr><th>(h-y)^2</th>{data.map(p=><td key={p.id}>{((predict(p.x,weights.w,weights.b)-p.y)**2).toFixed(3)}</td>)}</tr>
              <tr><th>삭제</th>{data.map(p=><td key={p.id}><button className="delete" title="데이터 삭제" onClick={()=>{setRunning(false);setData(d=>d.filter(q=>q.id!==p.id));}}>×</button></td>)}</tr>
            </tbody></table></div>
            <div className="add-row"><span>데이터 추가</span><label>x <input type="number" step="any" value={addX} onChange={e=>setAddX(e.target.value)}/></label><label>y <input type="number" step="any" value={addY} onChange={e=>setAddY(e.target.value)}/></label><button className="btn btn-blue" disabled={!addX.trim()||!addY.trim()||!Number.isFinite(Number(addX))||!Number.isFinite(Number(addY))} onClick={()=>addPoint(Number(addX),Number(addY))}>+ 추가</button><button className="btn btn-muted" onClick={()=>{setRunning(false);setData(initialData);nextId.current=11;reset();}}>기본값 복원</button></div>
          </section>
        </div>
      </main>
      <footer>React + TypeScript · 브라우저에서 직접 계산하는 데모 · Python/FastAPI 는 추후 연결 가능</footer>
    </div>;
}

export default App;
```